المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

École Nationale Supérieure de l'Intelligence Artificielle
et des Sciences des Données – Taroudant

Rapport d'Expertise en Sécurité Applicative

Erosion-Coastal Guard (Système de Monitoring)

Auditeurs :	Youssef Outakhrouft
	Youssef Oudra
Encadrant :	Pr. S. EL-ATEIF
Filière :	Sécurité IT et Confiance Numérique
Environnement :	http ://localhost :5000

Table des matières

1 Resume Executif **2**
1.1 Contexte . . . 2
1.2 Objectifs . . . 2
1.3 Resultats Globaux . . . 2
1.4 Niveau de Risque Global . . . 2

2 Methodologie de Test **2**
2.1 Approche . . . 2
2.2 Phases du Test . . . 3
2.3 Outils Utilises . . . 3

3 Perimetre des Tests **3**
3.1 Endpoints Testes . . . 3
3.2 Perimetre Exclu . . . 3

4 Tableau Recapitulatif des Vulnerabilites **3**

5 Findings Detailles **4**
5.1 SEC-01 – Injection SQL . . . 4
5.2 SEC-02 – Configuration CORS et SOP . . . 6
5.3 SEC-03 – Broken Access Control / IDOR . . . 7

6 Synthese du Niveau de Risque **9**

7 Plan d'Action et Remédiation **9**
7.1 Remédiation Technique du Contrôle d'Accès (SEC-03) . . . 9
7.2 Mesures de Durcissement (Hardening) . . . 10

8 Feuille de Route - Remédiation **10**
8.1 Plan d'Action Prioritaire . . . 10
8.2 Bonnes Pratiques à Maintenir . . . 10

9 Déclaration de Conformité **10**

Resume Executif

Contexte

Dans le cadre d'une démarche de sécurisation proactive, un test d'intrusion de type Boîte Grise a été conduit sur l'API du système de monitoring côtier **Erosion-Coastal Guard** (Flask/-Python). Ce test a été réalisé en avril 2026 par l'équipe Red Team composée de **Youssef Outakhrouft** et **Youssef Oudra**, étudiants à l'ENSIASD (École Nationale Supérieure de l'Intelligence Artificielle et des Sciences des Données – Taroudant), sur l'environnement local `http://localhost:5000`.

Objectifs

- Évaluer la résilience du système face aux attaques ciblant les données environnementales sensibles collectées (Agadir, Taghazout, etc.).
- Analyser les endpoints REST, la gestion des privilèges et la configuration du serveur Flask.
- Tester la robustesse face aux injections SQL, au contournement de contrôle d'accès, et aux problèmes de configuration CORS.
- Fournir des recommandations correctives concrètes et priorisées.

Resultats Globaux

Catégorie	Nombre	Détail
Vulnérabilités Critiques	1	SEC-03 : IDOR / Broken Access Control
Vulnérabilités Élevées	1	SEC-01 : Injection SQL (résistante mais configuration à surveiller)
Tests Non Vulnérables	1	SEC-02 : CORS
Total tests	3	

Niveau de Risque Global

NIVEAU DE RISQUE GLOBAL : CRITIQUE

Le système présente un risque global **CRITIQUE**. La vulnérabilité IDOR (SEC-03) permet à un attaquant authentifié de falsifier les données de surveillance en modifiant simplement un identifiant dans le corps de la requête, rendant les données scientifiques non fiables.

Action immédiate : Implémenter un contrôle d'accès basé sur la session utilisateur côté serveur et ne plus faire confiance aux identifiants fournis par le client.

Methodologie de Test

Approche

L'audit a suivi une approche de **Boîte Grise** en s'appuyant sur le référentiel OWASP Top 10 2021 :

- **Reconnaissance :** Analyse des routes API et des formats de données JSON.
- **Tests Automatisés :** Utilisation de scanners spécialisés (Sqlmap, CORScanner).
- **Analyse Manuelle :** Tests de logique métier et tentatives d'escalade de privilèges via Burp Suite et Curl.

Phases du Test

Phase	Nom	Description
1	Reconnaissance	Cartographie des endpoints, identification des technologies, analyse des en-têtes HTTP.
2	Mapping	Surface d'attaque, comportement des endpoints, analyse des flux de données.
3	Exploitation	Attaques : injection SQL, IDOR, contournement CORS, escalade de privilèges.
4	Documentation	Collecte des preuves, captures d'écran, rédaction du rapport.

Outils Utilises

Outil	Version	Usage
Sqlmap	Dernière version	Tests d'injection SQL automatisés
Burp Suite Community	Dernière version	Proxy HTTP, Repeater (injection manuelle)
CORScanner	Python	Détection de mauvaise configuration CORS
Curl	8.x	Tests manuels d'API et escalade de privilèges
Chrome DevTools	Intégré	Analyse des requêtes/réponses
Python 3	3.13	Scripts de tests personnalisés

Perimetre des Tests

Endpoints Testes

Méthode	Endpoint	Auth	Description
GET	/api/classification-history	Non require	Historique des classifications
GET	/api/audit-logs	Non require	Journaux d'audit système
POST	/api/releves	JWT requis	Envoi de relevés de monitoring
GET	/api/mesures	JWT requis	Mesures côtières
GET	/api/segments	JWT requis	Segments géographiques
GET	/api/alertes	JWT requis	Alertes actives

Perimetre Exclu

- Tests en environnement de production.
- Tests d'ingénierie sociale.
- Tests d'infrastructure réseau (hors application).
- Tests de déni de service volumétrique (DDoS réseau).

Tableau Recapitulatif des Vulnerabilites

ID	Vulnérabilité	Sévérité	Statut	CVSS
SEC-01	Injection SQL	Élevée	NON VULNÉRABLE	N/A
SEC-02	Mauvaise config CORS	Moyenne	NON VULNÉRABLE	N/A
SEC-03	Broken Access Control (IDOR)	CRITIQUE	**VULNÉRABLE**	9.1

Vecteurs CVSS

— **SEC-03 – CVSS 9.1 :** AV :N/AC :L/PR :L/UI :N/S :U/C :H/I :H/A :N
Accès réseau, faible complexité, privilèges faibles requis, impact total sur confidentialité et intégrité.

Findings Detailles

SEC-01 – Injection SQL

NON VULNÉRABLE

Sévérité :	Élevée (si vulnérable)
Statut :	**NON VULNÉRABLE**
CVSS :	N/A
Outil :	Sqlmap, Burp Suite Repeater

Description

L'injection SQL représente un risque majeur de fuite de base de données. Elle consiste à injecter des commandes SQL malveillantes dans les paramètres d'entrée pour manipuler les requêtes côté serveur.

Scénario d'Attaque

Tentative d'injection via le paramètre `limit` (type entier attendu) pour extraire la bannière du système de gestion de base de données (SGBD).

Preuve de Concept – Test 1

Commande exécutée :

```
python sqlmap.py -u "http://localhost:5000/api/classification-history?limit
    =50" --banner
```

Listing 1 – Test Sqlmap sur endpoint classification-history

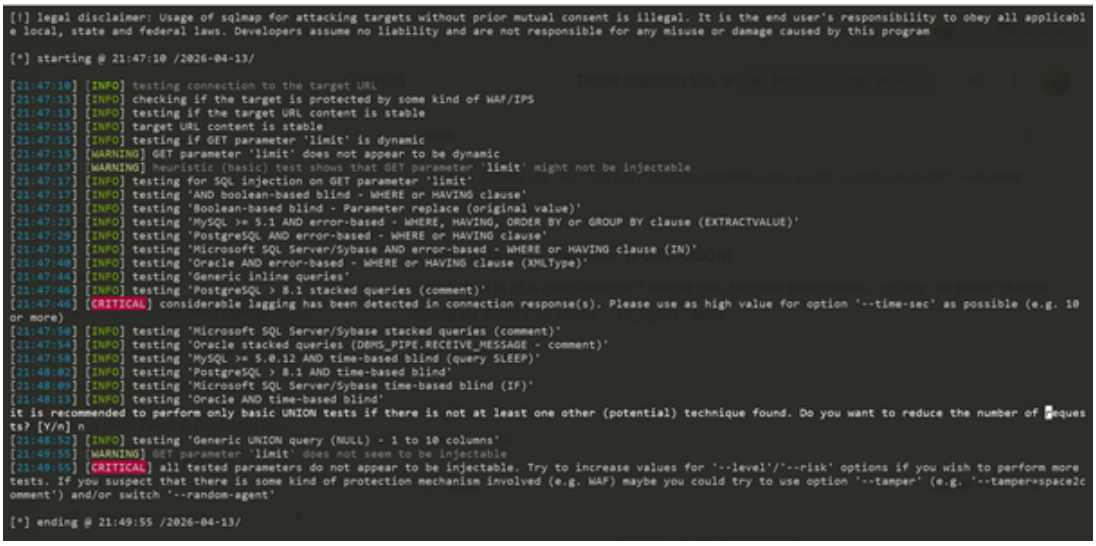

FIGURE 1 – Résultat Sqlmap – endpoint classification-history

Preuve de Concept – Test 2

Commande exécutée :

```
python sqlmap.py -u "http://localhost:5000/api/audit-logs?limit=10" --
    banner
```

Listing 2 – Test Sqlmap sur endpoint audit-logs

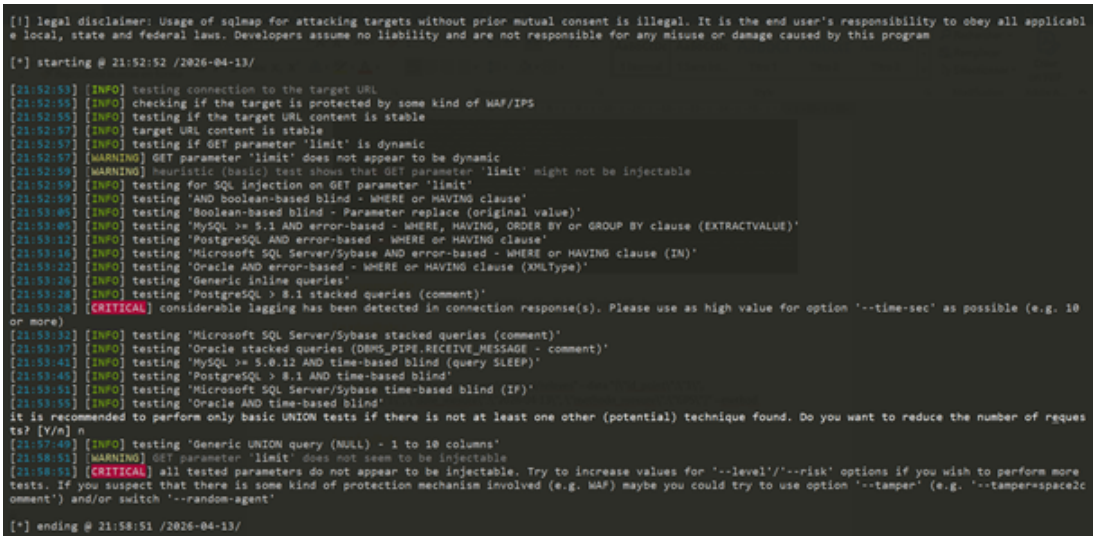

FIGURE 2 – Résultat Sqlmap – endpoint audit-logs

Preuve de Concept – Test 3 (POST)

Commande exécutée :

```
python sqlmap.py -u "http://localhost:5000/api/releves" --data "{\"id_point
    \":\"1\",␣\"id_agent\":\"1\",␣\"date_mesure\":\"2026-04-13\",␣\"
    methode_mesure\":\"GPS\"}" --method POST --banner
```

Listing 3 – Test Sqlmap sur endpoint POST /releves

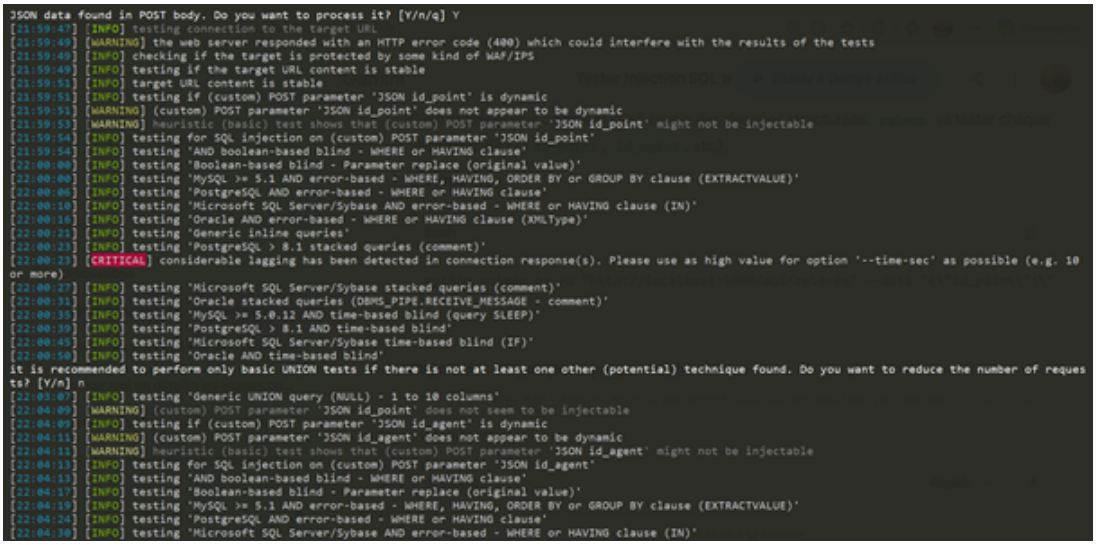

FIGURE 3 – Résultat Sqlmap – endpoint POST /releves

Observation

Le serveur rejette les caractères spéciaux (`'`, `-`, `;`) ou les traite comme des littéraux. Sqlmap n'a détecté aucune injection exploitable sur l'ensemble des paramètres testés.

Conclusion

NON VULNÉRABLE

L'implémentation utilise exclusivement l'interface DB-API de Python avec des **Prepared Statements**. Aucune injection SQL n'a été détectée. Maintenir l'utilisation des requêtes paramétrées et continuer les audits réguliers.

SEC-02 – Configuration CORS et SOP

NON VULNÉRABLE

Sévérité :	Moyenne (si vulnérable)
Statut :	**NON VULNÉRABLE**
CVSS :	N/A
Outil :	CORScanner, Burp Suite Repeater

Description

Le Cross-Origin Resource Sharing (CORS) est un mécanisme qui permet à une ressource de restreindre les requêtes provenant d'un domaine différent. Une mauvaise configuration (ex : `Access-Control-Allow-Origin: *`) permet à un site malveillant de lire les réponses de l'API via le navigateur d'une victime.

Scénario d'Attaque

Un attaquant héberge un site malveillant `attacker.com` qui tente de lire les réponses de l'API `http://localhost:5000/api/*` via une requête AJAX depuis le navigateur d'une victime authentifiée.

Preuve de Concept

```
<script>
    fetch('http://localhost:5000/api/mesures', {
        credentials: 'include'
    })
    .then(response => response.json())
    .then(data => console.log(data));
</script>
```

Listing 4 – Code malveillant tentant de lire l'API

Observation

Le serveur n'envoie pas l'en-tête `Access-Control-Allow-Origin: *`. Les navigateurs bloquent donc les lectures inter-domaines. Une réponse `401 Unauthorized` ou `403 Forbidden` est retournée sans en-têtes CORS permissifs.

Conclusion

NON VULNÉRABLE

La configuration CORS est sécurisée. Maintenir l'absence d'en-têtes CORS permissifs et envisager une configuration restrictive basée sur une liste blanche de domaines si nécessaire.

SEC-03 – Broken Access Control / IDOR

VULNÉRABILITÉ CRITIQUE

Sévérité :	**CRITIQUE**
Statut :	**VULNÉRABLE**
CVSS :	9.1 (AV :N/AC :L/PR :L/UI :N/S :U/C :H/I :H/A :N)
Outil :	Burp Suite Repeater, Curl

Description

La vulnérabilité IDOR (Insecure Direct Object Reference) survient lorsque l'application utilise une entrée fournie par l'utilisateur pour accéder directement à un objet sans vérifier les droits d'accès.

Analyse du Flux

Lors de l'envoi d'un relevé, le client fournit l'`id_agent` dans le corps de la requête JSON :

```
{
    "id_point": "1",
    "id_agent": "1",
    "date_mesure": "2026-04-13",
    "methode_mesure": "GPS",
    "valeur_erosion": "2.5"
}
```

Listing 5 – Requête POST /api/releves typique

Faille Critique

Le backend ne vérifie pas si l'`id_agent` dans le JSON correspond à l'identité réelle de l'utilisateur authentifié. Il n'existe aucun lien entre l'identifiant fourni dans la requête et la session authentifiée (via JWT).

Scénario d'Attaque

1. Un attaquant s'authentifie légitimement avec ses identifiants (agent ID 1).
2. Il capture la requête POST vers `/api/releves` via Burp Suite.
3. Il modifie la valeur de `id_agent` de 1 vers 2, 3, ou toute autre valeur.
4. Il rejoue la requête modifiée.
5. Le serveur accepte la requête et enregistre le relevé pour un autre agent.

Preuve de Concept

```
# Requ te l gitime avec id_agent=1 (agent authentifi )
curl -X POST http://localhost:5000/api/releves \
  -H "Authorization:␣Bearer␣<JWT_TOKEN_AGENT_1>" \
  -H "Content-Type:␣application/json" \
  -d '{"id_point":"1","id_agent":"1","date_mesure":"2026-04-13","
     methode_mesure":"GPS","valeur_erosion":"2.5"}'

# Requ te malveillante avec id_agent=2 (falsification)
curl -X POST http://localhost:5000/api/releves \
  -H "Authorization:␣Bearer␣<JWT_TOKEN_AGENT_1>" \
  -H "Content-Type:␣application/json" \
  -d '{"id_point":"1","id_agent":"2","date_mesure":"2026-04-13","
     methode_mesure":"GPS","valeur_erosion":"999.9"}'
```

Listing 6 – Exploitation IDOR via Curl

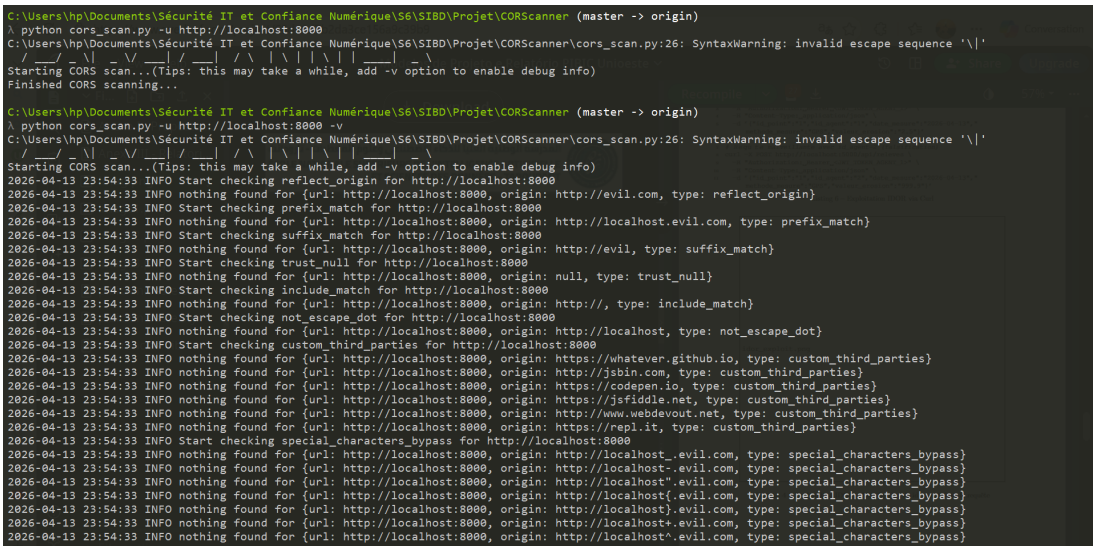

FIGURE 4 – Exploitation IDOR – Modification de id_agent dans la requête

Impact

- Un utilisateur malveillant peut falsifier les données de surveillance d'un autre site ou agent en changeant simplement un entier.
- Les données scientifiques deviennent non fiables et potentiellement manipulables.
- Perte d'intégrité des données de monitoring côtier.
- Impossible de tracer la source réelle des relevés frauduleux.
- Risque de décisions erronées basées sur des données falsifiées.

Conclusion

VULNÉRABILITÉ CRITIQUE

Il est impératif de ne plus faire confiance aux données d'identification envoyées dans le corps de la requête. Le contrôle d'accès doit être strictement basé sur l'identité extraite du token JWT (session utilisateur).

Synthese du Niveau de Risque

ID	Vulnérabilité	Sévérité Théorique	Risque Résiduel
SEC-01	Injection SQL	Élevée	Négligeable
SEC-02	Vol de données (CORS)	Moyenne	Faible
SEC-03	Broken Access Control (IDOR)	Critiques	Très Élevé

Niveau de Risque Global

CRITIQUE

Action immédiate requise : **SEC-03 – IDOR**

Plan d'Action et Remédiation

Remédiation Technique du Contrôle d'Accès (SEC-03)

Action Prioritaire – Délai : 24h

Il est impératif de ne plus faire confiance aux données d'identification envoyées dans le corps de la requête.

```python
from flask import request, g
from functools import wraps

def check_agent_identity():
    """
    V rifie que l'id_agent dans la requ te correspond
       l'utilisateur authentifi  via le token JWT.
    """
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            # Extraire l'ID de l'agent depuis le token JWT (session)
            authenticated_agent_id = g.current_user.id_agent

            # R cup rer l'id_agent de la requ te
            request_data = request.get_json()
            requested_agent_id = request_data.get('id_agent')

            # Comparaison stricte
            if str(authenticated_agent_id) != str(requested_agent_id):
                return {'error': 'Unauthorized␣access'}, 403

            return f(*args, **kwargs)
        return decorated_function
    return decorator

# Application sur l'endpoint sensible
@app.route('/api/releves', methods=['POST'])
@check_agent_identity()
def create_releve():
    # Logique m tier
    pass
```

Listing 7 – Correction recommandée – Contrôle basé sur la session

Mesures de Durcissement (Hardening)

1. **Validation de Schéma :** Utiliser Marshmallow pour définir un schéma strict (ex : rejeter tout champ inconnu ou non autorisé).
2. **Gestion des Erreurs :** Implémenter des messages d'erreur génériques. Actuellement, le système révèle l'existence d'IDs par ses réponses différenciées.
3. **Déploiement :** S'assurer que `FLASK_ENV` est réglé sur `production` pour désactiver le debugger interactif qui permet l'exécution de code à distance en cas de crash.
4. **Logging :** Mettre en place des logs détaillés pour toutes les tentatives d'accès non autorisées.
5. **Audit Régulier :** Planifier des tests d'intrusion réguliers après déploiement des correctifs.

Feuille de Route - Remédiation

ID	Vulnérabilité	Priorité	Effort	Délai
SEC-03	Broken Access Control (IDOR)	CRITIQUE	Faible	24h
SEC-01	Injection SQL	Informatif	N/A	Maintenir
SEC-02	Configuration CORS	Informatif	N/A	Maintenir

Plan d'Action Prioritaire

ACTIONS IMMÉDIATES – SOUS 24 HEURES

1. **[SEC-03]** Implémenter la vérification côté serveur entre `id_agent` du token JWT et `id_agent` du corps de la requête.
2. **[SEC-03]** Ne plus utiliser les identifiants envoyés par le client comme source de vérité.
3. **[SEC-03]** Tester la correction avec des scénarios de falsification d'identifiant.
4. **[Général]** Changer immédiatement tous les mots de passe par défaut.
5. **[Général]** Activer les logs de sécurité pour tracer toutes les requêtes.

Bonnes Pratiques à Maintenir

— **Injection SQL :** Continuer l'utilisation exclusive des Prepared Statements / ORM.
— **CORS :** Maintenir l'absence d'en-têtes CORS permissifs.
— **JWT :** Maintenir la politique de secrets forts et rotation tous les 90 jours.
— **Validation :** Toujours valider les entrées utilisateur.
— **Session :** Ne jamais faire confiance aux données client pour l'autorisation.

Déclaration de Conformité

Ce test d'intrusion a été réalisé dans un environnement de développement local (`http://localhost:5000`) avec l'autorisation expresse de l'équipe projet. Aucun système tiers n'a été impacté.

Ce rapport est strictement confidentiel, à l'usage exclusif de l'équipe projet et du corps enseignant de l'ENSIASD.

Youssef Outakhrouft
Étudiant - Red Team
ENSIASD - Taroudant
Avril 2026

Youssef Oudra
Étudiant - Red Team
ENSIASD - Taroudant
Avril 2026